|사물인터넷 실습|

Internet of Things and Laboratory

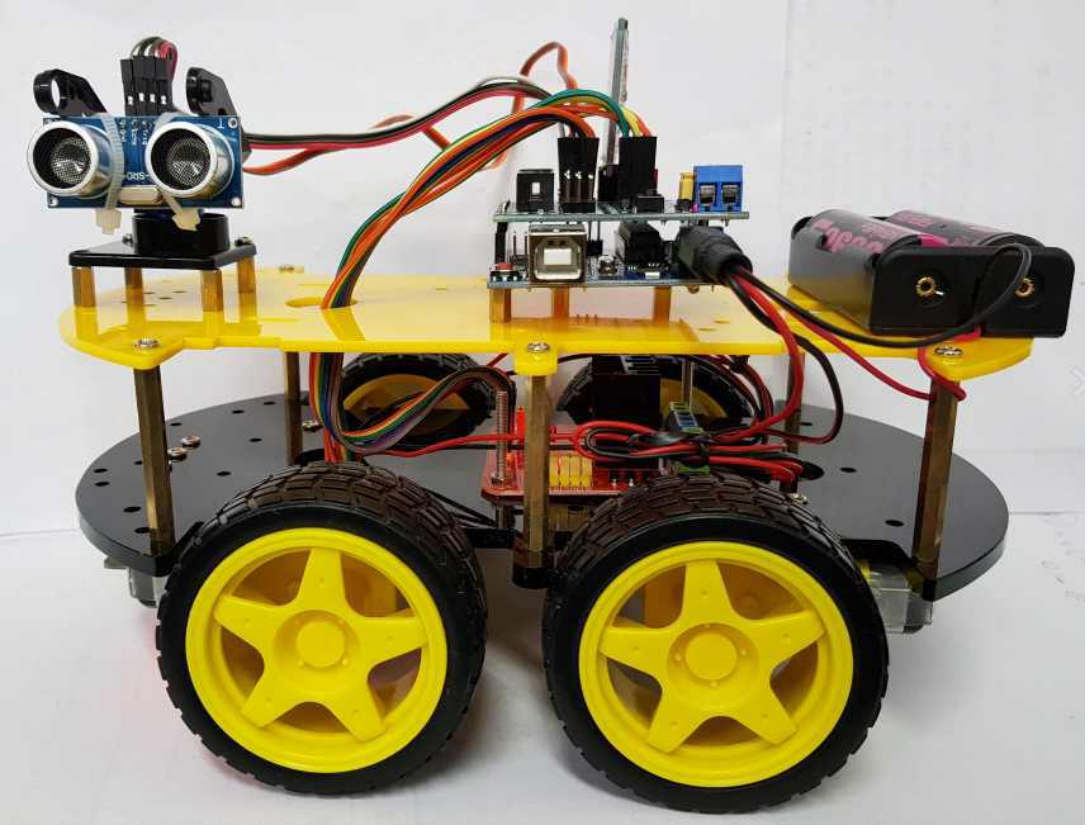
Arduino Car

강원대학교 컴퓨터공학과 권용훈

아두이노 카

차체 구성

- L298N 모터 드라이버
- DC 모터 4개
- 초음파 센서
- 서보 모터

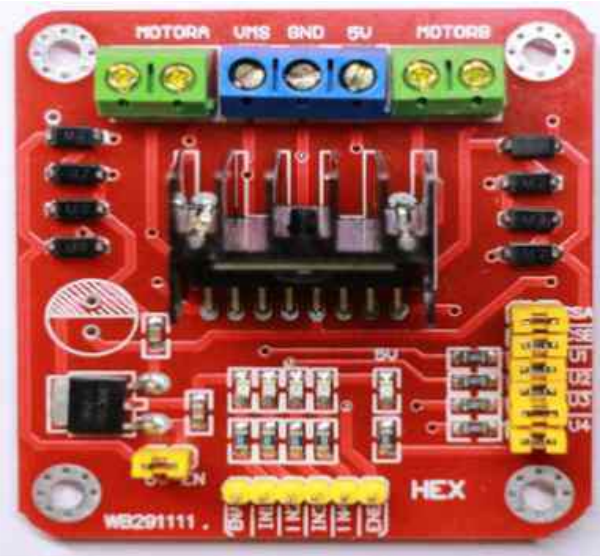


L298N 모터 드라이버

L298N 듀얼 H 브릿지

- 4개의 모터를 일괄적으로 제어하기 위한 드라이버

모터 좌측 2개 전원핀 모터 우측 2개



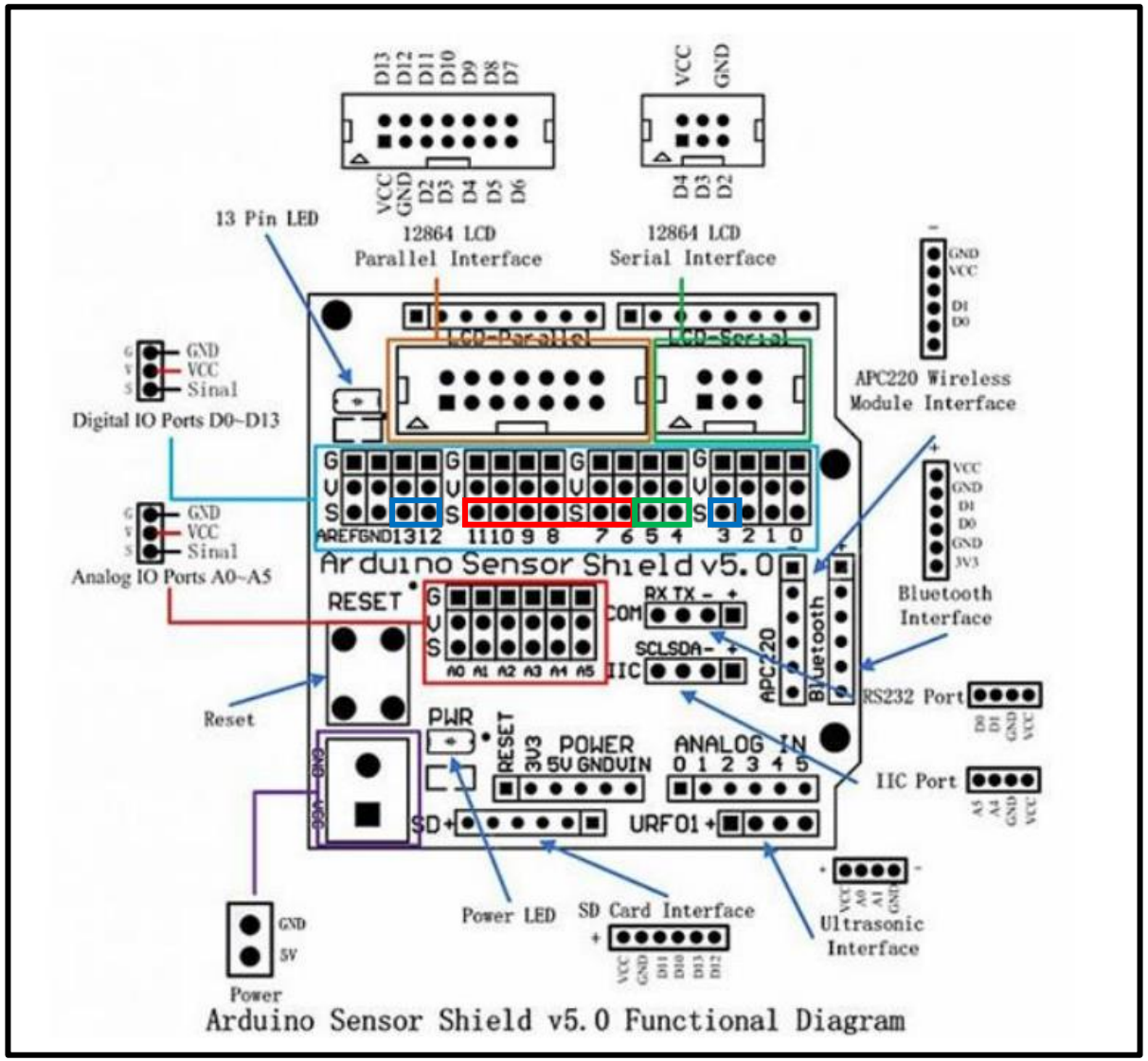
아두이노 디지털 핀

ENA (or ENB)	IN1 (or IN3)	IN2 (or IN4)	모터 A (or B)
High	High	Low	정방향 회전
High	Low	High	역방향 회전
High	High	High	정지
High	Low	Low	정지
Low	ANY	ANY	ANY

아두이노 쉴드

핀 번호

- 서보모터 : D3
- 초음파센서
  - Trigger: D5
  - Echo: D4
- 모터 드라이버
  - ENA: 11 ENB: 6
  - EN 1~4 : 10 ~ 7
- 블루투스 모듈
  - RX, TX - 12 ,13



아두이노 카 예제

```
#include <Servo.h>
int direction = 1;  // 1: 모터 회전 0: 서보 모터 회전
int speed = 100;
#define ENA 11
#define EN1 10
#define EN2 9
#define EN3 8
#define EN4 7
#define ENB 6
#define TRIGGER_PIN 5
#define ECHO_PIN 4
#define SERVO_PIN 3
Servo myservo;
int pos = 0;
void setup(){
  pinMode(ENA, OUTPUT); // ENA
  pinMode(EN1, OUTPUT); // EN1
  pinMode(EN2, OUTPUT); // EN2
  pinMode(ENB, OUTPUT); // ENB
  pinMode(EN3, OUTPUT); // EN3
  pinMode(EN4, OUTPUT); // EN4
  Serial.begin (9600);
  pinMode (TRIGGER_PIN, OUTPUT);
  pinMode (ECHO_PIN, INPUT);
  myservo.attach(SERVO_PIN);
}
```

```
void loop(){
  int duration, distance;
  digitalWrite(TRIGGER_PIN, HIGH);
  delayMicroseconds(1000);
  digitalWrite(TRIGGER_PIN, LOW);
  duration = pulseIn(ECHO_PIN, HIGH);
  distance = (duration/2) / 29.1;
  if (distance >= 4000 || distance <= 0) {
    Serial.print(distance);
    Serial.println("cm : Out of range");
  } else {
    Serial.print(distance);
    Serial.println(" cm");
  }
  delay(300);
if(direction == 1){   // 해당 코드를 기준으로 조정
    digitalWrite(EN1, LOW);
    digitalWrite(EN2, HIGH);
    analogWrite(ENA, speed);
    digitalWrite(EN3, LOW);
    digitalWrite(EN4, HIGH);
    analogWrite(ENB, speed);
  }else if(direction == 0){
    analogWrite(ENA, 0);
    analogWrite(ENB, 0);
    for(pos = 0; pos < 180; pos += 1){
      myservo.write(pos); delay(15);
    }
    for(pos = 180; pos>=1; pos-=1){
      myservo.write(pos); delay(15);
    }
  }
}
```

아두이노 실습 요령

차량 제어

- 차량마다 바퀴 연결 구조가 달라 예제 수행 후 조정 필요
  - 좌우가 다를 경우 한 쪽 바퀴의 신호 제어 코드를 반대로 수정
  - 양쪽이 다를 경우 양쪽의 신호 제어 코드 수정
  - 주행 시 차량이 한쪽으로 쏠릴 경우, 쏠리는 반대 방향의 바퀴의 속도를 낮춘다.
- 차량을 동작 시킬 때는 들고 있거나 바닥에 놓고 수행
- 블루투스: 각 차량의 번호에 맞는 모듈 이름과 핀번호가 설정
  - ex) 61번 차 - NAME: IoT0061 / PIN 0061
- 바퀴가 현저히 느려진 것이 느껴지면 배터리 교환
  - 완전 방전 시 재사용 불가
- 부품의 결합이 느슨해서 분해될 것 같은 경우 재결합 요청

과제 21

블루투스를 이용한 차량 제어

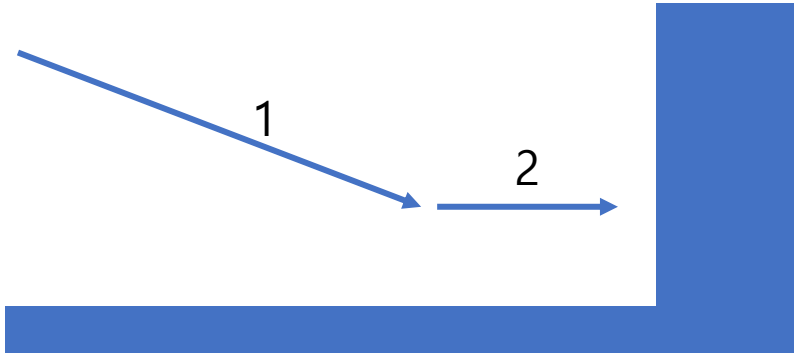
- 아두이노에는 1개의 스위치와 조이스틱 그리고 블루투스(마스터) 모듈이 연결된다.
- 스위치를 누르면 차량이 동작 가능해진다(시동을 건다).
  - 동작 가능한 상태에서 스위치를 다시 누르면 어떠한 입력을 넣어도 동작하지 않는다.
  - 동작 가능한 상태에서는 LED가 점등된다.
- 조이스틱은 상, 하,좌, 우 로 움직이면 차량이 해당 방향으로 움직인다.
  - 좌회전과 우회전은 제자리에서 수행된다.
- 조이스틱의 스위치를 누르면 서보 모터를 동작할 수 있다.
  - 이 때 조이스틱을 좌우로 움직이면 초음파를 탑재한 서보 모터가 좌우로 움직인다.
  - 조이스틱의 스위치를 다시 누를 경우 서보모터는 중앙 정렬되며 차량 동작 상태로 돌아온다.
- 최초에는 블루투스 신호가 정확히 전달되는지 Serial로 출력할 것

과제 22

갤럭시 탭을 이용한 차량 제어

- 21번에서 구현한 시스템을 사용한다.
- 메인 화면에 [실습 5] 버튼을 만들어 해당 기능을 구현한다.
- 스위치와 조이스틱의 기능을 버튼으로 구현한다.
- [전원] 버튼은 누르면 텍스트가 빨간색으로 변경되고 차량이 동작 가능해진다.
    - 동작하지 않는 상태는 검은색으로 표현된다.
- [전진], [후진], [좌회전], [우회전], [정지] 버튼을 구현한다.
- 초음파 센서의 값을 텍스트 뷰를 통하여 주기적으로 출력한다.
- [좌], [중앙], [우] 버튼을 만들고 서보모터를 제어한다. 각각 좌측이동, 중앙정렬, 우측이동 기능이다.

과제 23



평행주차

- 22번에서 구현한 시스템을 사용한다.
- 화면 아래에 [평행주차] 버튼을 구현하고, 누르면 다음과 같이 동작한다.
- 해당 버튼을 누르면 차량의 서보모터가 오른쪽으로 향한 뒤 고정된다.
- 차량이 전진하다가 벽에 가까워지면 좌측으로 살짝 틀어 벽면에 평행하게 정지한다.
- 정지한 뒤 서보모터는 중앙으로 정렬되고 벽을 따라 전진한다.
- 전방의 벽과 가까워지면 차량은 정지한다.

과제 23

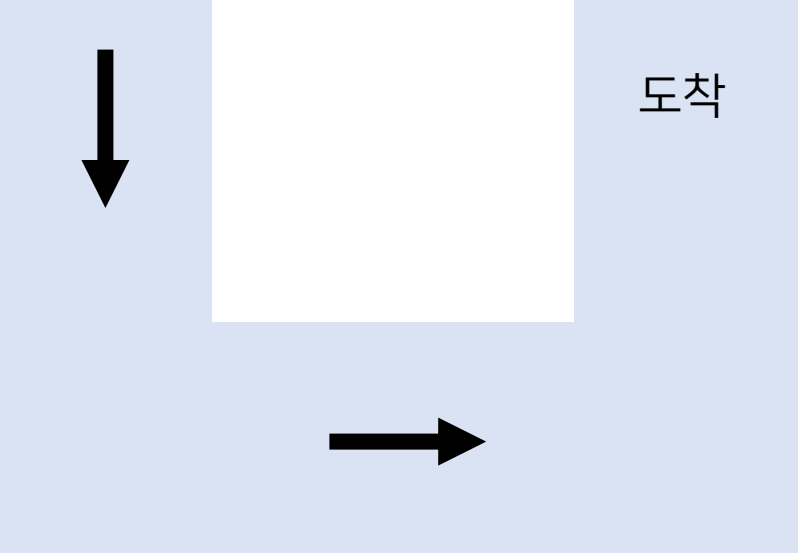
길 찾기

- 22번에서 구현한 시스템을 사용한다.
- 화면 아래에 [길 찾기] 버튼을 구현하고, 누르면 다음과 같이 동작한다.
- 차량이 막다른 길에 도달하여 해당 버튼을 누르면 초음파 센서와 서보 모터가 자동으로 동작한다.
- 서보모터가 회전하여 오른쪽과 왼쪽의 거리를 측정하고 길이 있는 쪽(막혀 있지 않은 쪽)으로 고개를 돌린다.
- 해당 버튼을 다시 누르면 해당 방향으로 차량이 회전하고 서보 모터는 중앙 정렬한다.

과제 25

ㄷ 코스 자율 주행

- 22번에서 구현한 시스템을 사용한다.
- 화면 아래에 [ㄷ 코스] 버튼을 구현하고, 누르면 다음과 같이 동작한다.
- ㄷ 자 형태의 코스를 주행하여 도착 지점에서 정지한다.
- 코너에 다다르면 좌우를 스캔하여 해당 방향으로 스스로 회전한다.
- 그리고 회전 이후에 전진하고, 다시 코너에서 회전한다.
- 도착 지점에는 정지한다.
- 도착과 시작 지점은 서로 바뀔 수 있다.



고생하셨습니다

Thank you!